

Parametric Probability Models, Information Theory, and Statistical Mechanics

A Unified Foundation of Logistic, Softmax, and Multi-Label Classification

Written in the Style of Goodfellow, Bengio, and Courville (*Deep Learning*)

Contents

1	The Machine Learning Paradigm and Statistical Mechanics Principles	2
1.1	Empirical Risk Minimization vs. Maximum Likelihood Estimation	2
1.2	Pathologies of Ordinary Least Squares (OLS) for Classification	2
1.3	Physical Analogy I: Two-State Quantum Systems and Phase Boundaries	3
2	Binary Logistic Regression: The Conditional Bernoulli Model	3
2.1	Exponential Families and the Natural Parameter (Logit Link)	3
2.2	The Sigmoid Activation Function: Properties and Symmetries	4
2.3	Maximum Likelihood and Cross-Entropy Loss	4
2.4	Information-Theoretic View: KL Divergence	5
2.5	Physical Analogy II: Thermodynamic Free Energy (ΔG) and Chemical Potential . .	5
3	Gradient Dynamics and Optimization Geometry	6
3.1	Derivation of the Gradient Vector	6
3.2	The Gradient Saturation Argument (Goodfellow 6.2.2)	6
3.3	Derivation of the Hessian Matrix and Proof of Convexity	7
3.4	Numerical Optimization: Gradient Descent and Newton-Raphson (IRLS)	8
4	Multinomial Logistic Regression: Softmax Activation	8
4.1	The Categorical Distribution Model	9
4.2	The Softmax Function	9
4.3	Physical Analogy III: Boltzmann Distribution and Partition Function (Z)	9
4.4	Categorical Cross-Entropy Loss and Softmax Gradient Derivation	10
4.5	Numerical Stability: The Log-Sum-Exp Trick	11
5	Multi-Label Logistic Regression: Non-Mutually Exclusive States	11
5.1	Why Softmax Fails for Multi-Label Tasks	11
5.2	Mathematical Formulation of Independent Sigmoids	11
5.3	Physical Analogy IV: Multi-Channel Spectroscopy and Non-Interfering Modes	12
6	Regularization, Priors, and Structural Risk Minimization	12
6.1	The Linearly Separable Data Pathology	12
6.2	L_2 Regularization (Weight Decay / Gaussian Prior)	12
6.3	L_1 Regularization (Lasso / Laplacian Prior)	12
6.4	Physical Analogy V: Harmonic Restoring Forces and Constraint Potentials	13
7	Evaluation Metrics and Detection Theory	13
7.1	The Pathology of Raw Accuracy	13
7.2	The Confusion Matrix and Signal Metrics	13
8	Production-Grade Vectorized NumPy Implementation	13

1 The Machine Learning Paradigm and Statistical Mechanics Principles

Supervised machine learning algorithms seek to approximate an unknown, underlying data-generating distribution $p_{\text{data}}(\mathbf{x}, y)$ given a finite set of N observed samples. Rather than attempting to model the full joint probability distribution $p(\mathbf{x}, y)$, discriminative classification models isolate the conditional distribution $p(y | \mathbf{x})$.

Definition 1.1 (Discriminative Learning Framework). Let $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ be a dataset of N independent and identically distributed (i.i.d.) observations sampled from p_{data} . Each input vector $\mathbf{x}^{(i)} \in \mathbb{R}^d$ represents a point in a d -dimensional feature space, and $y^{(i)}$ denotes a categorical or discrete label. A discriminative parametric model specifies a family of conditional probability distributions $p_{\text{model}}(y | \mathbf{x}; \boldsymbol{\theta})$, parameterized by a real-valued vector $\boldsymbol{\theta} \in \mathbb{R}^p$.

1.1 Empirical Risk Minimization vs. Maximum Likelihood Estimation

The primary objective in statistical learning is to minimize the expected risk (or generalization error) under the true distribution:

$$R(\boldsymbol{\theta}) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}} [L(y, f(\mathbf{x}; \boldsymbol{\theta}))] \quad (1)$$

where $L(\cdot, \cdot)$ is an element-wise loss metric. Because p_{data} is accessible only through empirical samples, we approximate expected risk via the **Empirical Risk Minimization (ERM)** framework:

$$\hat{R}_{\text{emp}}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N L(y^{(i)}, f(\mathbf{x}^{(i)}; \boldsymbol{\theta})) \quad (2)$$

When the loss function L is chosen to be the negative log-likelihood of the observed data, Empirical Risk Minimization becomes mathematically identical to **Maximum Likelihood Estimation (MLE)**:

$$\boldsymbol{\theta}_{\text{MLE}} = \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^N \log p_{\text{model}}(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \mathbb{E}_{(\mathbf{x}, y) \sim \hat{p}_{\text{data}}} [-\log p_{\text{model}}(y | \mathbf{x}; \boldsymbol{\theta})] \quad (3)$$

where $\hat{p}_{\text{data}}$ represents the empirical probability distribution defined by the training set.

1.2 Pathologies of Ordinary Least Squares (OLS) for Classification

Consider a binary classification scenario where $y \in \{0, 1\}$. A naive linear regression model hypothesis $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ attempts to fit target values directly using Mean Squared Error (MSE):

$$\mathcal{J}_{\text{MSE}}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^N \left(y^{(i)} - (\mathbf{w}^T \mathbf{x}^{(i)} + b) \right)^2 \quad (4)$$

Linear regression fails fundamentally in this setting due to three core mathematical and statistical breakdown modes:

1. **Violation of Kolmogorov's Probability Axioms:** The continuous linear function $\mathbf{w}^T \mathbf{x} + b$ maps onto $(-\infty, \infty)$. Interpreting $f(\mathbf{x})$ directly as a conditional probability $P(Y = 1 | \mathbf{x})$ violates basic probability theory whenever $\mathbf{w}^T \mathbf{x} + b < 0$ or $\mathbf{w}^T \mathbf{x} + b > 1$.
2. **Heteroscedastic Residual Variance:** For a Bernoulli-distributed target $Y | \mathbf{x} \sim \text{Bernoulli}(p)$, the conditional variance is a function of the mean:

$$\text{Var}(Y | \mathbf{x}) = p(\mathbf{x}) (1 - p(\mathbf{x})) = (\mathbf{w}^T \mathbf{x} + b) (1 - \mathbf{w}^T \mathbf{x} - b) \quad (5)$$

Because the variance changes for every input $\mathbf{x}$, the residual noise violates the homoscedasticity assumption required by the Gauss-Markov Theorem.

3. **Distortion from Outlier Leverage:** Minimizing MSE forces quadratic penalties on all residuals $(y^{(i)} - \hat{y}^{(i)})^2$. If an observation lies far on the correct side of the decision boundary (e.g., $y^{(i)} = 1$ and $\mathbf{w}^T \mathbf{x}^{(i)} + b = +10$), the squared residual is large $((1 - 10)^2 = 81)$. The model shifts its decision boundary away from the true partition simply to reduce errors on correctly classified points.

1.3 Physical Analogy I: Two-State Quantum Systems and Phase Boundaries

In statistical thermodynamics, a physical system limited to two discrete states (e.g., spin-up $s = +1$ vs. spin-down $s = -1$ in an Ising ferromagnet, or an empty $y = 0$ vs. occupied $y = 1$ electron site in a metal) does not change state along a linear path.

The probability of occupying a state depends on the balance between external energy biases and thermal energy fluctuations. The transition across a physical phase boundary behaves like an S-shaped activation function, where smooth microscopic thermal noise leads to sharp macroscale structural shifts.

2 Binary Logistic Regression: The Conditional Bernoulli Model

To model binary outputs correctly, we parameterize a conditional **Bernoulli distribution**:

$$P(Y = y \mid \mathbf{x}; \boldsymbol{\theta}) = \phi(\mathbf{x})^y (1 - \phi(\mathbf{x}))^{1-y}, \quad y \in \{0, 1\} \quad (6)$$

where $\phi(\mathbf{x}) \in (0, 1)$ dictates the probability parameter $P(Y = 1 \mid \mathbf{x})$.

2.1 Exponential Families and the Natural Parameter (Logit Link)

The Bernoulli distribution belongs to the Single-Parameter Exponential Family, defined as:

$$p(y \mid \eta) = h(y) \exp(\eta T(y) - A(\eta)) \quad (7)$$

where η is the natural (canonical) parameter, $T(y)$ is the sufficient statistic, $h(y)$ is the base measure, and $A(\eta)$ is the log-partition function.

Rewriting the Bernoulli distribution in exponential form:

$$\begin{aligned} p(y \mid \phi) &= \phi^y (1 - \phi)^{1-y} \\ &= \exp(\log[\phi^y (1 - \phi)^{1-y}]) \\ &= \exp(y \log \phi + (1 - y) \log(1 - \phi)) \\ &= \exp\left(y \log\left(\frac{\phi}{1 - \phi}\right) + \log(1 - \phi)\right) \end{aligned} \quad (8)$$

Equating this expression to the canonical exponential form yields:

- Base measure: $h(y) = 1$
- Sufficient statistic: $T(y) = y$
- Natural parameter: $\eta = \log\left(\frac{\phi}{1 - \phi}\right)$
- Log-partition function: $A(\eta) = -\log(1 - \phi) = \log(1 + e^\eta)$

Definition 2.1 (Log-Odds / Logit Link Function). The natural parameter η of the Bernoulli distribution defines the **Log-Odds (Logit)** of the event:

$$\text{logit}(\phi) = \log\left(\frac{\phi}{1 - \phi}\right) \quad (9)$$

In Generalized Linear Models (GLMs), we model this natural parameter as a linear combination of input features:

$$\eta = \log\left(\frac{\phi}{1 - \phi}\right) = \mathbf{w}^T \mathbf{x} + b \quad (10)$$

Solving directly for probability ϕ :

$$\frac{\phi}{1-\phi} = e^{\mathbf{w}^T \mathbf{x} + b} \implies \phi = \frac{e^{\mathbf{w}^T \mathbf{x} + b}}{1 + e^{\mathbf{w}^T \mathbf{x} + b}} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (11)$$

2.2 The Sigmoid Activation Function: Properties and Symmetries

The function mapping the linear score $z = \mathbf{w}^T \mathbf{x} + b \in (-\infty, \infty)$ to a valid probability $\phi \in (0, 1)$ is the **Logistic Sigmoid Function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z} \quad (12)$$

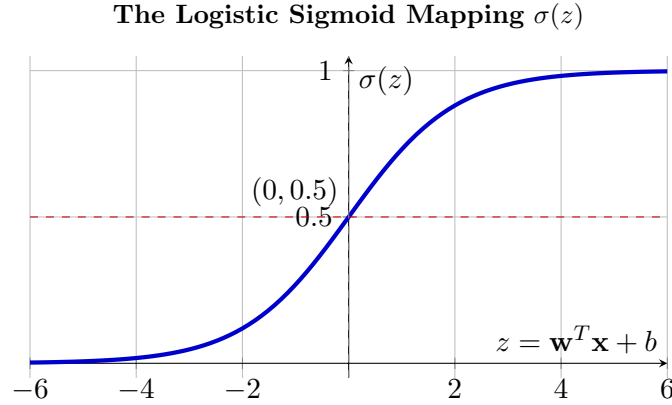


Figure 1: The logistic sigmoid function bounds unbounded real scores strictly to the probability domain $(0, 1)$.

Property 2.1 (Symmetry Under Reflection).

$$1 - \sigma(z) = 1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{e^z + 1} = \sigma(-z) \quad (13)$$

Property 2.2 (Derivative Algebra). The derivative of the sigmoid function expresses cleanly in terms of its output:

$$\begin{aligned} \frac{d\sigma(z)}{dz} &= \frac{d}{dz}(1 + e^{-z})^{-1} = -(1 + e^{-z})^{-2}(-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} = \left(\frac{1}{1 + e^{-z}} \right) \left(\frac{e^{-z}}{1 + e^{-z}} \right) \\ &= \sigma(z) (1 - \sigma(z)) \end{aligned} \quad (14)$$

2.3 Maximum Likelihood and Cross-Entropy Loss

Under the i.i.d. assumption, the total data likelihood over N samples given parameters $\boldsymbol{\theta} = [\mathbf{w}^T, b]^T$ is:

$$L(\boldsymbol{\theta}) = \prod_{i=1}^N P(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \prod_{i=1}^N \left(\hat{y}^{(i)} \right)^{y^{(i)}} \left(1 - \hat{y}^{(i)} \right)^{1-y^{(i)}} \quad (15)$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)} + b)$.

Taking the natural logarithm yields the **Log-Likelihood Function**:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^N \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \quad (16)$$

In machine learning, we frame training as minimizing a positive cost function $\mathcal{J}(\boldsymbol{\theta})$. Negating and normalizing the log-likelihood yields the **Binary Cross-Entropy (BCE) Loss** (also known as Log Loss):

$$\mathcal{J}_{\text{BCE}}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \quad (17)$$

2.4 Information-Theoretic View: KL Divergence

The cross-entropy loss arises naturally from information theory. Let $P(y | \mathbf{x})$ be the true empirical distribution defined by the target data, and let $Q(y | \mathbf{x}; \boldsymbol{\theta})$ be our model’s predicted distribution.

The **Kullback-Leibler (KL) Divergence** measures the statistical distance between P and Q :

$$D_{\text{KL}}(P \parallel Q) = \mathbb{E}_{y \sim P} \left[\log \left(\frac{P(y | \mathbf{x})}{Q(y | \mathbf{x}; \boldsymbol{\theta})} \right) \right] = \sum_{y \in \{0,1\}} P(y | \mathbf{x}) \log P(y | \mathbf{x}) - \sum_{y \in \{0,1\}} P(y | \mathbf{x}) \log Q(y | \mathbf{x}; \boldsymbol{\theta}) \quad (18)$$

The first term is the negative entropy $-H(P)$ of the true data distribution, which is constant with respect to model parameters $\boldsymbol{\theta}$. The second term is the **Cross-Entropy** $H(P, Q)$:

$$H(P, Q) = - \sum_{y \in \{0,1\}} P(y | \mathbf{x}) \log Q(y | \mathbf{x}; \boldsymbol{\theta}) \quad (19)$$

Thus, minimizing KL divergence is mathematically identical to minimizing cross-entropy, which in turn is equivalent to maximizing the log-likelihood under MLE:

$$\arg \min_{\boldsymbol{\theta}} D_{\text{KL}}(\hat{p}_{\text{data}} \parallel p_{\text{model}}) \equiv \arg \min_{\boldsymbol{\theta}} H(\hat{p}_{\text{data}}, p_{\text{model}}) \equiv \arg \min_{\boldsymbol{\theta}} \mathcal{J}_{\text{BCE}}(\boldsymbol{\theta}) \quad (20)$$

2.5 Physical Analogy II: Thermodynamic Free Energy (ΔG) and Chemical Potential

The log-odds link function maps directly to the thermodynamic relationship governing chemical reactions and phase equilibria. Consider a reversible two-state chemical reaction between Unbound (A) and Bound (B) molecular states:



The ratio of equilibrium populations defines the thermodynamic equilibrium constant K_{eq} :

$$K_{\text{eq}} = \frac{[B]}{[A]} = \frac{p_B}{1 - p_B} \quad (22)$$

According to statistical thermodynamics, K_{eq} is related to the change in standard Gibbs Free Energy (ΔG°) by:

$$\Delta G^\circ = -RT \log(K_{\text{eq}}) \implies \log \left(\frac{p_B}{1 - p_B} \right) = -\frac{\Delta G^\circ}{RT} \quad (23)$$

Comparing this to the logistic model equation:

$$\log \left(\frac{\phi}{1 - \phi} \right) = \mathbf{w}^T \mathbf{x} + b \equiv -\frac{\Delta G^\circ(\mathbf{x})}{k_B T} \quad (24)$$

The linear model score $z = \mathbf{w}^T \mathbf{x} + b$ acts as the normalized negative free energy difference of the reaction.

- The bias parameter b defines the baseline chemical potential difference of the system at standard conditions.
- The weight vector $\mathbf{w}$ quantifies how strongly external thermodynamic field parameters (such as temperature, voltage, or pressure) alter the free energy barrier to favor one physical state over another.

3 Gradient Dynamics and Optimization Geometry

Unlike ordinary linear regression, setting the gradient of the binary cross-entropy loss to zero ($\nabla_{\mathbf{w}} \mathcal{J} = \mathbf{0}$) yields a system of transcendental equations with no closed-form analytical solution. Optimizing the parameters requires numerical gradient algorithms.

3.1 Derivation of the Gradient Vector

Absorbing the bias term into the parameter vector $\mathbf{w} \leftarrow [\mathbf{w}^T, b]^T \in \mathbb{R}^{d+1}$ and extending inputs as $\mathbf{x} \leftarrow [\mathbf{x}^T, 1]^T \in \mathbb{R}^{d+1}$, the objective function for sample i is:

$$\mathcal{J}_i(\mathbf{w}) = -y^{(i)} \log \sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - (1 - y^{(i)}) \log (1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)})) \quad (25)$$

We apply the chain rule to compute the partial derivative with respect to weight component w_j :

$$\frac{\partial \mathcal{J}_i}{\partial w_j} = \frac{\partial \mathcal{J}_i}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial w_j} \quad (26)$$

Evaluating each term step-by-step:

1. Derivative of Loss with respect to prediction $\hat{y}^{(i)}$:

$$\frac{\partial \mathcal{J}_i}{\partial \hat{y}^{(i)}} = -\frac{y^{(i)}}{\hat{y}^{(i)}} + \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \quad (27)$$

2. Derivative of Sigmoid with respect to linear score $z^{(i)}$:

$$\frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} = \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \quad (28)$$

3. Derivative of Linear Score with respect to weight w_j :

$$\frac{\partial z^{(i)}}{\partial w_j} = x_j^{(i)} \quad (29)$$

Multiplying these three terms together causes the non-linear denominator $\hat{y}^{(i)}(1 - \hat{y}^{(i)})$ to cancel out completely:

$$\frac{\partial \mathcal{J}_i}{\partial w_j} = \left(\frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \right) \cdot (\hat{y}^{(i)}(1 - \hat{y}^{(i)})) \cdot x_j^{(i)} = (\hat{y}^{(i)} - y^{(i)})x_j^{(i)} \quad (30)$$

Averaging across all N data points gives the complete gradient vector:

$$\nabla_{\mathbf{w}} \mathcal{J}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)} = \frac{1}{N} X^T (\hat{\mathbf{y}} - \mathbf{y}) \quad (31)$$

where $X \in \mathbb{R}^{N \times (d+1)}$ is the design matrix, $\hat{\mathbf{y}} = [\hat{y}^{(1)}, \dots, \hat{y}^{(N)}]^T$, and $\mathbf{y} = [y^{(1)}, \dots, y^{(N)}]^T$.

3.2 The Gradient Saturation Argument (Goodfellow 6.2.2)

In Chapter 6 of *Deep Learning*, Goodfellow et al. emphasize a key theoretical reason why cross-entropy loss must be paired with sigmoid units rather than Mean Squared Error (MSE).

Consider an extreme wrong prediction where the true target is $y = 1$, but the model outputs $\hat{y} \approx 0$ (meaning $z = \mathbf{w}^T \mathbf{x} \ll 0$).

Case A: Sigmoid Paired with Mean Squared Error (MSE)

$$\mathcal{J}_{\text{MSE}} = \frac{1}{2}(\sigma(z) - y)^2 \quad (32)$$

Taking the gradient with respect to parameter vector $\mathbf{w}$:

$$\nabla_{\mathbf{w}} \mathcal{J}_{\text{MSE}} = (\sigma(z) - y) \cdot \sigma'(z) \cdot \mathbf{x} = (\sigma(z) - y) \cdot \sigma(z)(1 - \sigma(z)) \cdot \mathbf{x} \quad (33)$$

When $\hat{y} = \sigma(z) \rightarrow 0$, the activation derivative term $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ approaches 0. The gradient evaluates to:

$$\lim_{z \rightarrow -\infty} \nabla_{\mathbf{w}} \mathcal{J}_{\text{MSE}} = (0 - 1) \cdot (0) \cdot \mathbf{x} = \mathbf{0} \quad (34)$$

This phenomenon is called ****Gradient Saturation****. Even though the model is confidently wrong, the gradient vanishes entirely. Gradient descent stalls, preventing the parameters from updating.

Case B: Sigmoid Paired with Cross-Entropy Loss

As derived previously, combining cross-entropy loss with the sigmoid function cancels the activation derivative:

$$\nabla_{\mathbf{w}} \mathcal{J}_{\text{BCE}} = (\sigma(z) - y)\mathbf{x} \quad (35)$$

When $\hat{y} = \sigma(z) \rightarrow 0$ and $y = 1$, the gradient evaluates to:

$$\nabla_{\mathbf{w}} \mathcal{J}_{\text{BCE}} = (0 - 1)\mathbf{x} = -\mathbf{x} \neq \mathbf{0} \quad (36)$$

The logarithm inside the cross-entropy loss ****undoes the exponential saturation**** of the sigmoid ($\log e^{-z} = -z$). This eliminates gradient vanishing, ensuring strong gradient updates when the model makes confident mistakes.

3.3 Derivation of the Hessian Matrix and Proof of Convexity

To evaluate the local curvature of the loss surface, we compute the second-order partial derivatives of $\mathcal{J}(\mathbf{w})$:

$$\begin{aligned} H_{jk} &= \frac{\partial^2 \mathcal{J}}{\partial w_j \partial w_k} = \frac{\partial}{\partial w_k} \left(\frac{1}{N} \sum_{i=1}^N \left(\sigma(\mathbf{w}^T \mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \right) \\ &= \frac{1}{N} \sum_{i=1}^N \left(\frac{\partial}{\partial w_k} \sigma(\mathbf{w}^T \mathbf{x}^{(i)}) \right) x_j^{(i)} \\ &= \frac{1}{N} \sum_{i=1}^N \sigma(\mathbf{w}^T \mathbf{x}^{(i)}) \left(1 - \sigma(\mathbf{w}^T \mathbf{x}^{(i)}) \right) x_j^{(i)} x_k^{(i)} \end{aligned} \quad (37)$$

In matrix notation:

$$H = \nabla_{\mathbf{w}}^2 \mathcal{J}(\mathbf{w}) = \frac{1}{N} X^T R X \quad (38)$$

where $R \in \mathbb{R}^{N \times N}$ is a diagonal weighting matrix with entries $R_{ii} = \hat{y}^{(i)}(1 - \hat{y}^{(i)})$.

Theorem 3.1 (Convexity of Binary Cross-Entropy). The loss function $\mathcal{J}_{\text{BCE}}(\mathbf{w})$ is strictly convex if the design matrix X has full column rank.

Proof. Let $\mathbf{v} \in \mathbb{R}^{d+1} \setminus \{\mathbf{0}\}$ be an arbitrary non-zero vector. Quadratic form evaluation yields:

$$\mathbf{v}^T H \mathbf{v} = \mathbf{v}^T \left(\frac{1}{N} X^T R X \right) \mathbf{v} = \frac{1}{N} (X \mathbf{v})^T R (X \mathbf{v}) \quad (39)$$

Let $\mathbf{u} = X\mathbf{v} \in \mathbb{R}^N$. Expanding the diagonal matrix multiplication:

$$\mathbf{v}^T H \mathbf{v} = \frac{1}{N} \sum_{i=1}^N R_{ii} u_i^2 = \frac{1}{N} \sum_{i=1}^N \hat{y}^{(i)} (1 - \hat{y}^{(i)}) u_i^2 \quad (40)$$

Because $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)}) \in (0, 1)$ for all finite inputs, the diagonal terms are strictly positive ($R_{ii} > 0$). If X has full column rank, $X\mathbf{v} = \mathbf{0} \implies \mathbf{v} = \mathbf{0}$. Thus, for all $\mathbf{v} \neq \mathbf{0}$, $\mathbf{u} \neq \mathbf{0}$, which guarantees:

$$\mathbf{v}^T H \mathbf{v} > 0 \quad (41)$$

The Hessian matrix H is positive definite ($H \succ 0$) everywhere across parameter space. Consequently, the loss landscape is bowl-shaped and strictly convex, guaranteeing a unique global minimum. $\square$

3.4 Numerical Optimization: Gradient Descent and Newton-Raphson (IRLS)

Two primary optimization strategies are used to locate the minimum parameter vector $\mathbf{w}^*$:

1. First-Order Gradient Descent

Iterative parameter updates move opposite to the gradient scaled by learning rate $\alpha > 0$:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \nabla_{\mathbf{w}} \mathcal{J}(\mathbf{w}^{(t)}) = \mathbf{w}^{(t)} - \frac{\alpha}{N} X^T (\hat{\mathbf{y}}^{(t)} - \mathbf{y}) \quad (42)$$

2. Second-Order Optimization: Newton-Raphson / Iteratively Reweighted Least Squares

The Newton-Raphson method incorporates curvature information via the inverse Hessian matrix H^{-1} :

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - H^{-1} \nabla_{\mathbf{w}} \mathcal{J}(\mathbf{w}^{(t)}) \quad (43)$$

Substituting our gradient vector and Hessian matrix expressions:

$$\begin{aligned} \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - (X^T R X)^{-1} X^T (\hat{\mathbf{y}} - \mathbf{y}) \\ &= (X^T R X)^{-1} \left[(X^T R X) \mathbf{w}^{(t)} - X^T (\hat{\mathbf{y}} - \mathbf{y}) \right] \\ &= (X^T R X)^{-1} X^T R \left[X \mathbf{w}^{(t)} - R^{-1} (\hat{\mathbf{y}} - \mathbf{y}) \right] \end{aligned} \quad (44)$$

Defining the adjusted working response vector $\mathbf{z}_{\text{adj}} = X \mathbf{w}^{(t)} - R^{-1} (\hat{\mathbf{y}} - \mathbf{y})$ gives:

$$\mathbf{w}^{(t+1)} = (X^T R X)^{-1} X^T R \mathbf{z}_{\text{adj}} \quad (45)$$

This update equation is mathematically identical to solving a Weighted Least Squares (WLS) linear regression problem at each iteration. For this reason, second-order optimization of logistic regression is called **Iteratively Reweighted Least Squares (IRLS)**.

4 Multinomial Logistic Regression: Softmax Activation

When the target variable spans $K > 2$ mutually exclusive categories ($y \in \{1, 2, \dots, K\}$), binary logistic regression generalizes to **Multinomial Logistic Regression** (Softmax Regression).

4.1 The Categorical Distribution Model

We represent target labels as one-hot encoded vectors $\mathbf{y} \in \{0, 1\}^K$, where $y_k = 1$ if the observation belongs to class k , and $y_j = 0$ for all $j \neq k$. The output is modeled using a **Categorical (Multinomial) Distribution**:

$$P(\mathbf{Y} = \mathbf{y} \mid \mathbf{x}; W) = \prod_{k=1}^K P(Y = k \mid \mathbf{x}; W)^{y_k} = \prod_{k=1}^K (p_k)^{y_k} \quad (46)$$

Instead of a single weight vector $\mathbf{w}$, the model uses a weight matrix $W = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K] \in \mathbb{R}^{(d+1) \times K}$ to generate a vector of K unnormalized real-valued logits $\mathbf{z} \in \mathbb{R}^K$:

$$z_k = \mathbf{w}_k^T \mathbf{x} + b_k \quad (47)$$

4.2 The Softmax Function

To convert unnormalized logits $\mathbf{z}$ into a valid probability vector $\mathbf{p}$ satisfying $p_k > 0$ and $\sum_{k=1}^K p_k = 1$, we apply the **Softmax Function**:

$$p_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \frac{e^{\mathbf{w}_k^T \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x} + b_j}} \quad (48)$$

Property 4.1 (Translation Invariance). Adding a constant scalar c to all logit components leaves the softmax probabilities unchanged:

$$\text{softmax}(\mathbf{z} + c\mathbf{1})_k = \frac{e^{z_k + c}}{\sum_{j=1}^K e^{z_j + c}} = \frac{e^{z_k} e^c}{e^c \sum_{j=1}^K e^{z_j}} = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} = \text{softmax}(\mathbf{z})_k \quad (49)$$

This property reveals an over-parameterization in raw softmax representations, which is resolved during practical implementation using numerical shifts.

4.3 Physical Analogy III: Boltzmann Distribution and Partition Function (Z)

The Softmax function is mathematically identical to the **Boltzmann Distribution** governing statistical state occupancies in thermal physics.

Consider a physical microcanonical ensemble containing K non-interacting energy states, where each state k possesses an energy E_k . At thermal equilibrium with a heat reservoir at temperature T , the probability P_k of finding the system in quantum state k is:

$$P_k = \frac{e^{-E_k/k_B T}}{\sum_{j=1}^K e^{-E_j/k_B T}} = \frac{e^{-E_k/k_B T}}{Z} \quad (50)$$

where k_B is the Boltzmann constant, and $Z = \sum_{j=1}^K e^{-E_j/k_B T}$ is the **Canonical Partition Function**.

Comparing the two formulations:

$$z_k \equiv -\frac{E_k}{k_B T} \quad (51)$$

- Each unnormalized logit score z_k represents the normalized negative energy level of physical state k . Higher scores correspond to lower, more stable physical energy configurations.
- The denominator $\sum_{j=1}^K e^{z_j}$ acts as the **Canonical Partition Function (Z)**. It sums over all accessible state weights, normalizing the probability distribution across all competing physical outcomes.

4.4 Categorical Cross-Entropy Loss and Softmax Gradient Derivation

The negative log-likelihood across N samples under a categorical distribution defines the **Categorical Cross-Entropy Loss**:

$$\mathcal{J}_{\text{CCE}}(W) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log p_k^{(i)} \quad (52)$$

To compute the gradient vector for weight column $\mathbf{w}_j$, we first compute the Jacobian of the softmax function $\left(\frac{\partial p_k}{\partial z_j}\right)$:

$$\frac{\partial p_k}{\partial z_j} = \frac{\partial}{\partial z_j} \left(\frac{e^{z_k}}{\sum_{m=1}^K e^{z_m}} \right) \quad (53)$$

Evaluating the two possible cases:

1. When $k = j$ (Derivative of numerator and denominator):

$$\frac{\partial p_j}{\partial z_j} = \frac{e^{z_j} (\sum_m e^{z_m}) - e^{z_j} e^{z_j}}{(\sum_m e^{z_m})^2} = \frac{e^{z_j}}{\sum_m e^{z_m}} - \left(\frac{e^{z_j}}{\sum_m e^{z_m}} \right)^2 = p_j(1 - p_j) \quad (54)$$

2. When $k \neq j$ (Derivative of denominator only):

$$\frac{\partial p_k}{\partial z_j} = \frac{0 - e^{z_k} e^{z_j}}{(\sum_m e^{z_m})^2} = - \left(\frac{e^{z_k}}{\sum_m e^{z_m}} \right) \left(\frac{e^{z_j}}{\sum_m e^{z_m}} \right) = -p_k p_j \quad (55)$$

Combining these using Kronecker's delta δ_{kj} :

$$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j) \quad (56)$$

Now, applying the chain rule to compute the gradient of the loss for a single sample i with respect to score logit z_j :

$$\begin{aligned} \frac{\partial \mathcal{J}_i}{\partial z_j} &= - \sum_{k=1}^K \frac{y_k}{p_k} \frac{\partial p_k}{\partial z_j} \\ &= - \sum_{k=1}^K \frac{y_k}{p_k} (p_k(\delta_{kj} - p_j)) \\ &= - \sum_{k=1}^K y_k(\delta_{kj} - p_j) \\ &= - \left(y_j(1 - p_j) + \sum_{k \neq j} y_k(-p_j) \right) \\ &= -y_j + y_j p_j + \sum_{k \neq j} y_k p_j \\ &= -y_j + p_j \sum_{k=1}^K y_k \end{aligned} \quad (57)$$

Because $\mathbf{y}$ is one-hot encoded, exactly one element equals 1, so $\sum_{k=1}^K y_k = 1$. The expression simplifies cleanly to:

$$\frac{\partial \mathcal{J}_i}{\partial z_j} = p_j - y_j \quad (58)$$

Multiplying by $\frac{\partial z_j}{\partial \mathbf{w}_j} = \mathbf{x}$ and accumulating across all N samples in matrix notation ($X \in \mathbb{R}^{N \times (d+1)}$, $P \in \mathbb{R}^{N \times K}$, $Y \in \mathbb{R}^{N \times K}$):

$$\nabla_W \mathcal{J}_{\text{CCE}}(W) = \frac{1}{N} X^T (P - Y) \quad (59)$$

4.5 Numerical Stability: The Log-Sum-Exp Trick

In digital computing, evaluating e^{z_k} directly for large logits (e.g., $z_k = 1000$) causes floating-point overflow ('inf'). Conversely, large negative logits (e.g., $z_k = -1000$) cause floating-point underflow ('0'), leading to division-by-zero errors ($\log 0 = -\infty$).

To make computations numerically stable, we exploit the translation invariance property by setting scalar $c = -\max_j z_j$:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k - \max_j z_j}}{\sum_{j=1}^K e^{z_j - \max_j z_j}} \quad (60)$$

When evaluating log-probabilities directly inside cross-entropy, we use the ****Log-Sum-Exp (LSE)**** formulation:

$$\log \text{softmax}(\mathbf{z})_k = z_k - \text{LSE}(\mathbf{z}) = z_k - \left(m + \log \sum_{j=1}^K e^{z_j - m} \right) \quad (61)$$

where $m = \max_j z_j$. This guarantees that the largest exponent is $e^0 = 1$, preventing numerical overflow.

5 Multi-Label Logistic Regression: Non-Mutually Exclusive States

In ****Multi-Label Classification****, an observation can simultaneously possess zero, one, or multiple active categorical attributes. The target vector is a binary mask $\mathbf{y} \in \{0, 1\}^K$, where multiple entries y_k can equal 1 at the same time.

5.1 Why Softmax Fails for Multi-Label Tasks

Softmax forces class predictions into competition through a shared normalization denominator $\sum e^{z_j}$. Increasing logit score z_1 increases probability p_1 , which automatically forces down the probabilities $p_2, \dots, p_K$ of all other classes.

In multi-label problems, attributes are non-mutually exclusive. Detecting attribute 1 should not artificially depress the probability of detecting attribute 2.

5.2 Mathematical Formulation of Independent Sigmoids

To handle non-competing targets, we drop the softmax normalization denominator and model K ****independent conditional Bernoulli distributions**** using a single parameter matrix $W \in \mathbb{R}^{(d+1) \times K}$:

$$p_k = P(Y_k = 1 \mid \mathbf{x}; \mathbf{w}_k) = \sigma(z_k) = \frac{1}{1 + e^{-(\mathbf{w}_k^T \mathbf{x} + b_k)}} \quad (62)$$

The multi-label objective function sums K individual Binary Cross-Entropy losses across all targets:

$$\mathcal{J}_{\text{MultiLabel}}(W) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left[y_k^{(i)} \log p_k^{(i)} + (1 - y_k^{(i)}) \log(1 - p_k^{(i)}) \right] \quad (63)$$

Because each output node operates independently, the gradient with respect to parameter matrix W retains the same matrix structure:

$$\nabla_W \mathcal{J}_{\text{MultiLabel}}(W) = \frac{1}{N} X^T (P - Y) \quad (64)$$

where $P_{ik} = \sigma(\mathbf{w}_k^T \mathbf{x}^{(i)} + b_k)$.

5.3 Physical Analogy IV: Multi-Channel Spectroscopy and Non-Interfering Modes

Multi-label classification mirrors **multi-channel absorption spectroscopy** (such as Infrared or Mass Spectroscopy).

Consider analyzing a chemical sample containing an unknown mixture of organic molecules. The presence of an O-H Hydroxyl group produces an absorption band at 3300 cm^{-1} , while a C=O Carbonyl group produces an absorption peak at 1700 cm^{-1} .

- The presence of a Hydroxyl group ($y_1 = 1$) does not prevent or require the simultaneous presence of a Carbonyl group ($y_2 = 1$).
- Each absorption mode operates as an independent quantum oscillator filter. The multi-label model runs K independent parallel detectors on the same input feature spectrum, evaluating each vibrational mode without constraining the sum of detected peaks.

6 Regularization, Priors, and Structural Risk Minimization

6.1 The Linearly Separable Data Pathology

If a training dataset is linearly separable, a decision boundary hyperplane exists that correctly classifies all training points. Under unregularized MLE, the objective attempts to force predictions to exact ground truth values: $\hat{y}^{(i)} \rightarrow 1$ for $y^{(i)} = 1$, and $\hat{y}^{(i)} \rightarrow 0$ for $y^{(i)} = 0$.

Because $\sigma(z) \rightarrow 1$ only as $z \rightarrow +\infty$, the optimizer drives parameter norms to infinity ($\|\mathbf{w}\|_2 \rightarrow \infty$). The sigmoid function steepens into a discontinuous step function, producing severe overfitting and numerical instability. Regularization is required to keep parameters bounded.

6.2 L_2 Regularization (Weight Decay / Gaussian Prior)

L_2 regularization appends a quadratic penalty on weight magnitudes to the loss function:

$$\mathcal{J}_{\text{Ridge}}(\mathbf{w}) = \mathcal{J}_{\text{data}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 = \mathcal{J}_{\text{data}}(\mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \quad (65)$$

From a Bayesian perspective, L_2 regularization corresponds to Maximum A Posteriori (MAP) estimation under a zero-mean **Gaussian Prior** on parameters $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \frac{1}{\lambda} I)$:

$$\mathbf{w}_{\text{MAP}} = \arg \max_{\mathbf{w}} \left[\sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \mathbf{w}) + \log p(\mathbf{w}) \right] \quad (66)$$

The regularized gradient update shrinks weights toward zero at each step:

$$\nabla_{\mathbf{w}} \mathcal{J}_{\text{Ridge}} = \frac{1}{N} X^T (\hat{\mathbf{y}} - \mathbf{y}) + \lambda \mathbf{w} \quad (67)$$

6.3 L_1 Regularization (Lasso / Laplacian Prior)

L_1 regularization appends an absolute value norm penalty:

$$\mathcal{J}_{\text{Lasso}}(\mathbf{w}) = \mathcal{J}_{\text{data}}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 = \mathcal{J}_{\text{data}}(\mathbf{w}) + \lambda \sum_{j=1}^d |w_j| \quad (68)$$

This corresponds to MAP estimation under a zero-mean **Laplacian Prior** $p(\mathbf{w}) \propto \exp(-\lambda \|\mathbf{w}\|_1)$. Because the absolute value function has sharp corners at $w_j = 0$, L_1 regularization forces non-essential feature weights to exact zeros, performing automated feature selection.

6.4 Physical Analogy V: Harmonic Restoring Forces and Constraint Potentials

Regularization operates like adding a physical potential energy constraint to a dynamic system.

- **L_2 Regularization** acts like a 3D **Harmonic Oscillator Potential Well** $V(\mathbf{w}) = \frac{1}{2}k\|\mathbf{w}\|^2$. As parameter particles $\mathbf{w}$ drift away from the origin under data gradient forces, the potential well exerts a continuous linear restoring force $\mathbf{F} = -\lambda\mathbf{w}$ pulling them back toward the origin.
- **L_1 Regularization** acts like a constant **V-shaped Friction Potential** $V(\mathbf{w}) = \gamma \sum |w_j|$. It exerts a constant force magnitude regardless of distance, trapping weak feature parameters at exact zero coordinates.

7 Evaluation Metrics and Detection Theory

7.1 The Pathology of Raw Accuracy

Evaluating classification performance using raw accuracy ($\frac{\text{Correct}}{\text{Total}}$) fails on imbalanced datasets. Consider a medical dataset with 1,000,000 samples where only 100 patients have a rare disease ($y = 1$). A dummy classifier that continuously outputs "Healthy" ($\hat{y} = 0$) achieves 99.99% accuracy while failing completely as a diagnostic tool.

7.2 The Confusion Matrix and Signal Metrics

Predictions are organized into a 2×2 **Confusion Matrix**:

	Model Predicts Positive ($\hat{y} = 1$)	Model Predicts Negative ($\hat{y} = 0$)
Actual Positive ($y = 1$)	True Positive (TP)	False Negative (FN)
Actual Negative ($y = 0$)	False Positive (FP)	True Negative (TN)

Definition 7.1 (Precision / Signal Purity). Fraction of positive model claims that are genuine signals:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (69)$$

Definition 7.2 (Recall / Detector Efficiency). Fraction of total real signals captured by the model:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (70)$$

Definition 7.3 (F1-Score / Harmonic Mean). The harmonic mean balancing Precision (P) and Recall (R):

$$\text{F1-Score} = 2 \cdot \frac{P \cdot R}{P + R} = \frac{2}{\frac{1}{P} + \frac{1}{R}} \quad (71)$$

The harmonic mean is mathematically analogous to **Reduced Mass** ($\mu = \frac{m_1 m_2}{m_1 + m_2}$) or **Parallel Electrical Resistance** ($R_{\text{eq}} = \frac{R_1 R_2}{R_1 + R_2}$). If either Precision or Recall approaches zero, the combined F1-Score collapses to zero.

8 Production-Grade Vectorized NumPy Implementation

The following Python class implements Binary, Multi-Class (Softmax), and Multi-Label logistic regression models, complete with numerical stability protections and evaluation metrics.

```

1 import numpy as np
2
3 class UniversalLogisticEngine:
4     """
5     Unified Classification Framework implementing Binary, Multi-Class (
6     Softmax),
7     and Multi-Label Logistic Regression with L2 Regularization.
8     """
9     def __init__(self, learning_rate: float = 0.1, max_epochs: int = 1000,
10                  l2_penalty: float = 0.01, tolerance: float = 1e-6):
11         self.lr = learning_rate
12         self.max_epochs = max_epochs
13         self.l2_penalty = l2_penalty
14         self.tol = tolerance
15         self.W = None # Parameter matrix ((d+1) x K)
16
17     @staticmethod
18     def _add_bias(X: np.ndarray) -> np.ndarray:
19         """Appends column of ones for bias term: X_b = [X, 1]."""
20         return np.hstack([X, np.ones((X.shape[0], 1))])
21
22     @staticmethod
23     def _sigmoid(Z: np.ndarray) -> np.ndarray:
24         """Numerically stable sigmoid function."""
25         Z_clipped = np.clip(Z, -25.0, 25.0)
26         return 1.0 / (1.0 + np.exp(-Z_clipped))
27
28     @staticmethod
29     def _softmax(Z: np.ndarray) -> np.ndarray:
30         """Numerically stable softmax function using translation shift."""
31         Z_shifted = Z - np.max(Z, axis=1, keepdims=True)
32         exp_Z = np.exp(Z_shifted)
33         return exp_Z / np.sum(exp_Z, axis=1, keepdims=True)
34
35     def fit_binary(self, X: np.ndarray, y: np.ndarray):
36         """Fits single-output Binary Logistic Regression model."""
37         X_b = self._add_bias(X)
38         N, d_bias = X_b.shape
39         y_col = y.reshape(-1, 1)
40         self.W = np.zeros((d_bias, 1))
41
42         for epoch in range(self.max_epochs):
43             p = self._sigmoid(X_b @ self.W)
44             error = p - y_col
45
46             # Gradient: dL/dW = (1/N) * X^T * error + lambda * W
47             grad = (1 / N) * (X_b.T @ error) + self.l2_penalty * self.W
48             self.W -= self.lr * grad
49
50             if np.linalg.norm(grad) < self.tol:
51                 break
52
53     def fit_multiclass(self, X: np.ndarray, y: np.ndarray, num_classes: int):
54         """Fits Multi-Class Softmax Regression model (Mutually Exclusive)."""
55
56         X_b = self._add_bias(X)
57         N, d_bias = X_b.shape
58         K = num_classes

```

```

58     # One-hot target encoding
59     Y_onehot = np.zeros((N, K))
60     Y_onehot[np.arange(N), y] = 1.0
61     self.W = np.zeros((d_bias, K))
62
63     for epoch in range(self.max_epochs):
64         P = self._softmax(X_b @ self.W)
65         error = P - Y_onehot
66
67         grad = (1 / N) * (X_b.T @ error) + self.l2_penalty * self.W
68         self.W -= self.lr * grad
69
70         if np.linalg.norm(grad) < self.tol:
71             break
72
73     def fit_multilabel(self, X: np.ndarray, Y: np.ndarray):
74         """Fits Multi-Label Regression model (Non-Mutually Exclusive)."""
75         X_b = self._add_bias(X)
76         N, d_bias = X_b.shape
77         K = Y.shape[1]
78         self.W = np.zeros((d_bias, K))
79
80         for epoch in range(self.max_epochs):
81             P = self._sigmoid(X_b @ self.W)
82             error = P - Y
83
84             grad = (1 / N) * (X_b.T @ error) + self.l2_penalty * self.W
85             self.W -= self.lr * grad
86
87             if np.linalg.norm(grad) < self.tol:
88                 break
89
90     def predict_proba(self, X: np.ndarray) -> np.ndarray:
91         """Returns continuous probability matrix P."""
92         X_b = self._add_bias(X)
93         if self.W.shape[1] == 1:
94             return self._sigmoid(X_b @ self.W)
95         return self._softmax(X_b @ self.W)
96
97     def predict(self, X: np.ndarray, threshold: float = 0.5) -> np.ndarray:
98         """Returns hard class label predictions."""
99         probs = self.predict_proba(X)
100         if self.W.shape[1] == 1:
101             return (probs >= threshold).astype(int).flatten()
102         return np.argmax(probs, axis=1)
103
104
105     def evaluate_classifier(y_true: np.ndarray, y_pred: np.ndarray):
106         """Computes Confusion Matrix, Precision, Recall, and F1-Score."""
107         tp = np.sum((y_true == 1) & (y_pred == 1))
108         fp = np.sum((y_true == 0) & (y_pred == 1))
109         fn = np.sum((y_true == 1) & (y_pred == 0))
110         tn = np.sum((y_true == 0) & (y_pred == 0))
111
112         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
113         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
114         f1 = 2 * precision * recall / (precision + recall) if (precision +
115         recall) > 0 else 0.0
116
117         return {"TP": tp, "FP": fp, "FN": fn, "TN": tn,

```

```

117         "Precision": precision, "Recall": recall, "F1-Score": f1}
118
119
120 if __name__ == "__main__":
121     np.random.seed(42)
122
123     # 1. Validation Run: Binary Model
124     X_bin = np.random.randn(200, 3)
125     y_bin = (X_bin[:, 0] + X_bin[:, 1] > 0).astype(int)
126
127     model_bin = UniversalLogisticEngine(learning_rate=0.2, max_epochs=1000)
128     model_bin.fit_binary(X_bin, y_bin)
129     preds_bin = model_bin.predict(X_bin)
130
131     metrics = evaluate_classifier(y_bin, preds_bin)
132     print("== BINARY MODEL EVALUATION METRICS ==")
133     for k, v in metrics.items():
134         print(f"{k:<12}: {v}")
135
136     print("\n" + "="*40 + "\n")
137
138     # 2. Validation Run: Multi-Class Softmax Model
139     X_mc = np.random.randn(300, 3)
140     y_mc = np.random.choice([0, 1, 2], size=300)
141
142     model_mc = UniversalLogisticEngine(learning_rate=0.2, max_epochs=1000)
143     model_mc.fit_multiclass(X_mc, y_mc, num_classes=3)
144     preds_mc = model_mc.predict(X_mc)
145
146     acc_mc = np.mean(preds_mc == y_mc)
147     print(f"== MULTI-CLASS SOFTMAX ACCURACY: {acc_mc * 100:.2f}% ==")

```

Listing 1: Vectorized NumPy implementation of Logistic, Softmax, and Multi-Label classification.